Frequency Analyzer Algorithms Explained

A plain-language guide to what each algorithm finds, how it works, why it is used, and why its result matters.

> **Who this is for:** A reader who has not seen this project before and may not have a background in software-defined radio or digital signal processing.

The project in one paragraph

The application receives complex in-phase and quadrature samples, usually called IQ samples, from an SDR. Those samples describe a radio signal as it changes over time. The software converts the samples into a spectrum, where the horizontal axis is frequency and the vertical axis is signal level. It then searches that spectrum for carriers and peaks, measures bandwidth and power, maintains historical traces, applies calibration, and places markers on useful points.

Algorithms covered

- Spectrum estimation using a Hann-windowed FFT.
- Adaptive carrier-band detection.
- Detection of multiple distinct spectral peaks.
- Sub-bin estimation of the strongest signal frequency.
- Noise-floor estimation.
- Occupied-bandwidth measurement.
- Channel-power integration.
- Maximum hold, minimum hold, and power averaging.
- Power calibration from dBFS to dBm.
- HackRF frequency-axis correction.
- Automatic peak tracking and marker peak search.

The explanations describe the current implementation in the source code. They intentionally avoid features that are only ordinary user interface operations, such as snapping a marker to the nearest bin.

Before the algorithms: four simple ideas

1. IQ samples are time-domain measurements.

An SDR does not directly hand the program a list of radio stations. It supplies a stream of numerical samples. Each sample contains magnitude and phase information for the received radio waveform.

2. A spectrum is a frequency-domain view.

The spectrum reorganizes the same information by frequency. A strong tone appears as a peak. A digitally modulated transmission usually appears as a wider raised band.

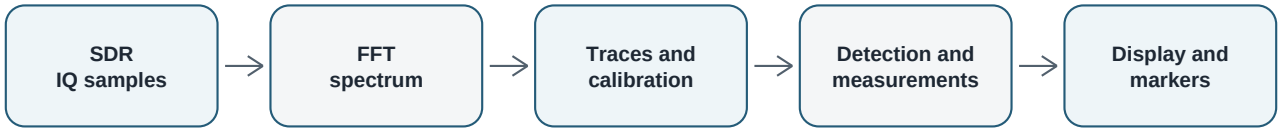
3. An FFT bin is one small frequency slot.

The 4096-point FFT divides the sampled bandwidth into 4096 slots. Each slot has a center frequency and a measured level. The approximate bin spacing, also reported as resolution bandwidth, is sample rate divided by 4096.

4. dBFS and dBm are different.

dBFS says how large a digital sample is compared with the ADC's full scale. dBm describes physical RF power relative to one milliwatt. The program can only report dBm when a valid device calibration is available.

Overall processing flow



Each stage adds information while preserving the original raw dBFS trace.

Important distinction: Carrier detection, multi-peak detection, and the measurement panel do related jobs, but they are not duplicates. A carrier is a whole occupied frequency band. A peak is one locally high point. A measurement is a single numerical summary of the current spectrum.

1. Spectrum estimation with a Hann-windowed FFT

Implementation: backend/dsp.py

WHAT IT FINDS	WHY WE USE IT
The frequencies present in the incoming IQ block and the relative level of each frequency.	The SDR supplies time-domain samples, while a spectrum analyzer must show frequency-domain information.
SIGNIFICANCE	
Every detector and measurement in the application depends on this spectrum. If the spectrum is inaccurate or unstable, all later results will also be inaccurate.	

How it works, step by step

- **Remove the mean:** The average value of the IQ block is subtracted. This acts as a simple DC blocker and reduces a false spike at the exact center of the display.
- **Apply a Hann window:** The block is gently reduced toward zero at both ends. This prevents the sudden block boundaries from spreading one signal into many neighboring frequency bins.
- **Compute the FFT:** The Fast Fourier Transform separates the time-domain samples into frequency bins.
- **Shift the result:** The negative-frequency half is moved to the left and the positive-frequency half to the right, placing the receiver's center frequency in the middle.
- **Normalize:** The FFT magnitude is divided by the Hann window's coherent gain so that the displayed level of a tone is not reduced merely because a window was used.
- **Convert to dBFS:** The linear magnitude becomes 20 log10(magnitude). Values below the numerical floor are clamped to -140 dBFS so logarithms remain finite.
- **Crop the displayed span:** If the requested display span is smaller than the sample rate, bins outside the requested range are removed.

Simple example

Imagine that the receiver contains one pure tone 125 kHz above its center frequency. The FFT places most of that tone's energy near the bin whose frequency is center + 125 kHz. The Hann window keeps the tone from appearing as a wide artificial smear.

> **Why the Hann window matters:** Real signals do not normally start and stop exactly at the boundaries of a 4096-sample block. Without windowing, the artificial jump between the last and first sample creates spectral leakage that can hide weak nearby signals or create misleading shoulders.

2. Adaptive carrier-band detection

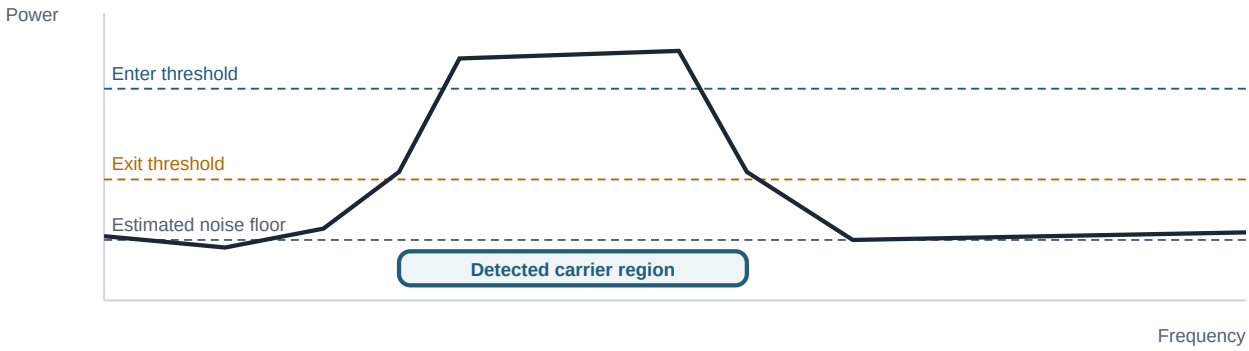
Implementation: backend/carrier_detection.py

WHAT IT FINDS	WHY WE USE IT
Sustained occupied frequency bands, including their left edge, right edge, center bin, strongest bin, peak level, noise estimate, and a confidence value.	A wide modulated transmission is not well described by one peak. We need to identify the complete occupied region while rejecting random noise spikes and receiver passband artifacts.

SIGNIFICANCE
This algorithm lets the application count carriers and draw overlays that align with actual band edges. It is the main signal-presence decision algorithm in the project.

Core idea: use two thresholds instead of one

A single threshold causes unstable edges. A slightly noisy point may make a band repeatedly appear, disappear, or change width. The detector therefore uses a high threshold to prove that a real carrier exists and a lower threshold to follow that carrier out to its weaker edges. This is called hysteresis.



The high line is the **enter threshold**. A carrier must contain a long, strong core above it. The lower line is the **exit threshold**. Once a strong core exists, the region can extend down to this lower line so that the rising and falling shoulders are retained.

Step 1: smooth the trace

A short moving average reduces bin-to-bin fluctuations. The default window adapts to the number of displayed bins and is kept odd so it has a clear center. Edge padding repeats the end values instead of inserting zero dB, because zero-padding a negative-dB spectrum would manufacture very large false edge peaks.

Step 2: estimate the background robustly

The detector first finds the 60th percentile of the smoothed trace and keeps values at or below it. The median of those lower values becomes the background noise estimate. Strong occupied bins are therefore prevented from pulling the estimated floor upward.

2. Adaptive carrier-band detection - decision process

Step 3: measure how rough the noise is

The detector looks at the change between neighboring smoothed bins. It uses the median absolute deviation, or MAD, of those changes to estimate random floor variation. MAD is used because it is less sensitive to a few extreme values than standard deviation.

Step 4: create adaptive thresholds

- Enter level = noise floor + the larger of 10 dB or 6 times the estimated noise variation.
- Exit level = noise floor + the larger of 3 dB or 3 times the estimated noise variation.

On a clean receiver, the configured 10 dB and 3 dB margins dominate. On a rough receiver, the variation-based terms raise the thresholds automatically and reduce false detections.

Step 5: require a sustained strong core

The algorithm finds consecutive runs above the lower threshold. Inside each run it requires at least 50 consecutive smoothed bins above the high threshold. A single narrow spike therefore cannot become a carrier.

Step 6: reject incomplete edge regions

A region touching the first or last FFT bin is rejected. The detector cannot see background on both sides of such a region, so it cannot reliably claim that both band edges were measured. This also removes many passband-edge artifacts.

Step 7: refine the edges on unsmoothed data

Smoothing is useful for making the decision but can shift an edge. The detector therefore returns to the original unsmoothed spectrum. A valid edge needs three consecutive bins above the exit level. This keeps one noisy bin from pulling the boundary outward.

Step 8: describe the result

- The peak bin is the strongest original bin inside the detected region.
- The center bin is halfway between the two measured edges. It is the geometric center of the region, not necessarily the peak.
- Confidence describes how far the peak rises above the enter threshold and is limited to the range 0 to 1.
- Nearby regions can optionally be merged when their gap is below a configured number of bins. The current default does not merge gaps.

> **Significance:** This combination of robust noise estimation, hysteresis, run-length qualification, and unsmoothed edge refinement is designed to keep broad low-prominence noise humps from being labeled as carriers while preserving the beginning and end of a real digital transmission.

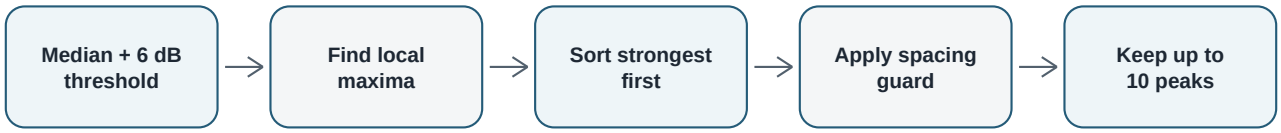
3. Detection of multiple distinct peaks

Implementation: backend/peak.py

WHAT IT FINDS	WHY WE USE IT
Up to ten strong, separate local peaks in the current live spectrum.	Several transmissions may be visible at once. Simply taking the largest bin would report only one of them, while returning every small bump would mostly report noise or several points from the same signal.

SIGNIFICANCE
The peak list gives a compact set of notable signals for measurement and display without filling the result with closely spaced duplicates.

How it works



- **Adaptive threshold:** The median amplitude of the live spectrum is calculated, and 6 dB is added. The median makes the threshold follow the current displayed floor.
- **Local maximum test:** A candidate must be higher than the bin on its left, at least as high as the bin on its right, and above the threshold.
- **Ranking:** Candidates are sorted from strongest to weakest.
- **Guard spacing:** After one peak is accepted, another candidate is rejected if it is closer than roughly 1 percent of the displayed bin count. The guard is never smaller than three bins.
- **Limit:** Processing stops after ten peaks have been accepted.

Simple example

Suppose one transmitter creates a cluster of five high bins and a second transmitter creates another cluster farther away. The local maximum test finds the tops of both clusters. The spacing guard keeps only one representative from each cluster, so the result is two useful peaks rather than ten almost identical entries.

> **Difference from carrier detection:** A peak is a point. A carrier is a region. A wide flat-topped carrier may have several small local maxima, but carrier detection should still return one occupied band.

4. Strongest-frequency estimation with sub-bin interpolation

Implementation: backend/measurements.py

WHAT IT FINDS	WHY WE USE IT
The frequency of the strongest live signal with a result that can fall between FFT-bin centers.	A plain maximum search is limited to the fixed bin grid. The true signal can lie between two bins, making the reported frequency jump in whole-bin steps.
SIGNIFICANCE	
Parabolic interpolation provides a smoother and usually more accurate peak-frequency readout without increasing the FFT size or processing another sample block.	

How it works

- Find the strongest amplitude bin.
- Read that bin and its immediate left and right neighbors.
- Fit the top of a parabola through those three logarithmic amplitude values.
- Use the parabola's vertex to estimate how far the real peak lies from the center bin.
- Multiply that fractional-bin shift by the bin spacing and add it to the center-bin frequency.

If the strongest bin is at an array edge, the three points form a flat or unusable curve, or the calculated shift is unreasonable, the algorithm safely returns the original bin frequency.

Simple example

With 488 Hz bin spacing, the strongest bin might be centered at 100.125000 MHz. If its two neighbors suggest that the peak is 0.3 bins to the right, the estimated frequency becomes approximately 100.125146 MHz instead of being forced to the bin center.

5. Displayed noise-floor estimation

Implementation: backend/measurements.py

WHAT IT FINDS	WHY WE USE IT
A representative per-bin background level across the displayed span.	The average can be pulled upward by strong transmissions. The median stays representative as long as less than about half of the displayed bins are dominated by signals.
SIGNIFICANCE	
The result gives the reader a simple indication of receiver background and signal visibility. It also provides context for interpreting peak height.	

How it works

All displayed live-spectrum amplitudes are sorted conceptually, and the middle value is used. In practice NumPy computes the median directly. This is intentionally simple and robust.

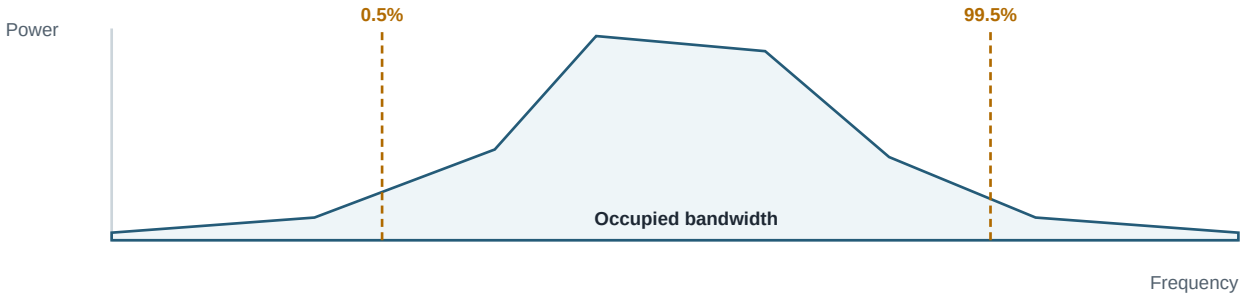
> **Do not confuse the two noise estimates:** The measurement panel uses the median of all displayed bins. Carrier detection uses a more conservative lower-60-percent median plus a MAD-based roughness estimate because it must make detection decisions.

6. Occupied-bandwidth measurement

Implementation: backend/measurements.py

WHAT IT FINDS	WHY WE USE IT
The frequency interval containing the middle 99 percent of all displayed spectral power.	A signal's width should be based on its distributed power rather than on one arbitrary amplitude threshold.

SIGNIFICANCE
Occupied bandwidth summarizes how much spectrum the received energy uses and is a standard way to describe modulated signals.



How it works

- Convert each logarithmic bin value back to linear power using 10 raised to amplitude divided by 10.
- Add the linear powers to obtain total displayed power.
- Build a cumulative sum from the lowest displayed frequency to the highest.
- Find the first bin where cumulative power reaches 0.5 percent of the total.
- Find the first bin where cumulative power reaches 99.5 percent.
- Subtract the lower frequency from the upper frequency.

Why 0.5 percent and 99.5 percent?

Removing 0.5 percent from each tail leaves the middle 99 percent. Very small distant components therefore have less influence than they would in a simple first-nonzero-to-last-nonzero width.

> **Interpretation limit:** The current implementation uses every displayed bin, including noise and unrelated transmissions. A weak signal in a wide span can therefore produce a large occupied-bandwidth value. It is not yet a measurement restricted to one detected carrier region.

7. Channel-power integration

Implementation: backend/measurements.py

WHAT IT FINDS	WHY WE USE IT
The sum of linear power across the complete displayed span.	Decibel values cannot be added directly. Each bin must first be converted to linear power.
SIGNIFICANCE	
This gives one total-power number for the displayed channel or span. When calibration is valid, the result is presented as calibrated dBm.	

The same linear powers used for occupied bandwidth are summed. The result is converted back with 10 log10(total power). A very small positive floor prevents the logarithm of zero.

> **Current scope:** The algorithm integrates the complete displayed span. It does not currently use detected carrier edges as channel boundaries.

8. Maximum hold, minimum hold, and power averaging

Implementation: backend/trace.py

WHAT IT FINDS	WHY WE USE IT
For every frequency bin: the highest observed level, the lowest valid observed level, and the average power over all processed frames.	A live trace shows only one instant. Intermittent, drifting, or noisy signals are easier to understand when their history is retained.
SIGNIFICANCE	
These traces reveal brief transmissions, long-term minima, and stable average spectral shapes that a single frame can miss.	

Maximum hold

For each frequency bin, compare the new live value with the stored value and keep the larger one. A short signal remains visible after it disappears from the live trace.

Minimum hold

For each frequency bin, keep the smaller valid value. Samples at or near the artificial -140 dBFS DSP floor are ignored because they represent numerical cancellation or underflow rather than a real receiver measurement. On initialization, invalid bins use the median of valid bins as a safe fallback.

Running power average

The live dB value is first converted to linear power. The new running mean is updated using:

> **Running-mean rule:** new average = old average + (new power - old average) / new frame count

The average is then converted back to dB for display. This update needs only the current average and frame count, so the program does not have to store every previous spectrum.

Why averaging must happen in linear power

dB is logarithmic. Directly averaging dB values does not represent average physical power and can suppress noise-like digital modulation. The implementation therefore averages linear detector power and only uses dB for presentation.

Simple example

If one bin is measured once at a high level and once at a low level, maximum hold keeps the high value, minimum hold keeps the low value, and average calculates the mean of their linear powers. These three results answer different questions about the same history.

9. Power-calibration offset selection

Implementation: backend/power_calibration.py

WHAT IT FINDS	WHY WE USE IT
The correction value that must be added to a raw dBFS reading to produce an estimated physical input power in dBm.	An SDR's raw digital level depends on receiver gain, frequency response, and hardware variation. Relabeling dBFS as dBm without calibration would give a believable-looking but incorrect power.

SIGNIFICANCE
The calibration logic makes absolute power readouts traceable to measured device behavior and deliberately falls back to honest dBFS when calibration is not valid.

Decision order

- **Validity guard:** If the current VGA gain is below or above the characterized range, no dBm result is produced.
- **Per-VGA table:** This has highest priority because it directly stores the solved offset for measured gain values.
- **Frequency table:** If no VGA table exists, the algorithm interpolates a base offset between neighboring frequency calibration points, then subtracts the current VGA gain.
- **Constant model:** If the response was sufficiently uniform, one base offset is used and the VGA gain is subtracted.
- **Legacy constant:** An older already-solved offset is accepted for backward compatibility.

Piecewise-linear interpolation

When the requested gain or frequency falls between two measured calibration points, the correction is estimated along the straight line joining those points. Outside the table, the nearest endpoint is used rather than extrapolating into an unmeasured range.

Simple example

Suppose the frequency table gives a base offset of -10 dB at 100 MHz and -20 dB at 200 MHz. At 150 MHz the interpolated base is -15 dB. With 30 dB VGA gain, the applied offset is -45 dB. A raw reading of -20 dBFS would therefore be reported as -65 dBm.

> **Safety significance:** Returning no calibration outside a measured range is an important design choice. It prevents the interface from presenting fabricated absolute power merely because a numerical formula is available.

10. HackRF frequency-axis correction

Implementation: backend/acquisition.py

WHAT IT FINDS	WHY WE USE IT
A corrected center frequency after accounting for a constant tuning error and an error that grows in proportion to frequency.	A hardware oscillator can have both a fixed offset and a parts-per-million scale error. Without correction, every displayed frequency can be shifted from its true RF value.
SIGNIFICANCE	
Correcting the center frequency improves the accuracy of the entire frequency axis, peak readouts, carrier edges, marker positions, and bandwidth placement.	

The two-term model

> **Frequency error:** error in Hz = fixed error in Hz + driver frequency x PPM error x 0.000001

> **Corrected frequency:** calibrated frequency = driver frequency - calculated error

Why two terms?

- The fixed term describes an error that remains approximately constant at all tuned frequencies.
- The PPM term describes proportional oscillator error. For the same PPM value, the error in hertz becomes larger as the tuned frequency increases.

Sign convention

Calibration records observed frequency minus reference frequency. A positive result means the analyzer displays too high, so the software subtracts that error from the driver frequency.

Simple example

At a 1 GHz driver frequency, a fixed +2 kHz error and a +1 PPM error produce a total error of +3 kHz. The calibrated center becomes 999.997 MHz.

Backward compatibility

If the two-term calibration is not present, the HackRF acquisition path can still apply the older single frequency-axis offset.

> **Scope:** The explicit fixed-plus-PPM correction is currently implemented in the HackRF acquisition path. USRP and Pluto use their driver-reported configured center frequencies.

11. Automatic peak tracking and marker peak search

Implementation: frontend/renderer.py and webapp/static/app.js

WHAT IT FINDS	WHY WE USE IT
The strongest displayed point on the user-selected trace, such as live, maximum hold, minimum hold, or average.	The user often wants to see or place a marker on the most prominent visible feature immediately, without manually searching across thousands of bins.
SIGNIFICANCE	
This makes the analysis interactive: the automatic red marker follows the strongest point, and peak search can place a normal marker there.	

Desktop behavior

The desktop renderer ignores non-finite frequency or amplitude values, then performs a maximum search across the selected trace. The automatic marker is moved to that bin. Manual marker peak search uses the same basic maximum-bin idea.

Web behavior

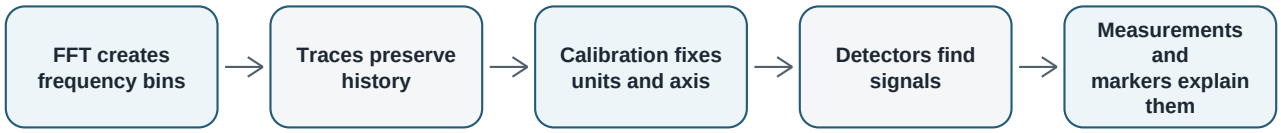
The browser's reusable visible-peak function scans from the first visible bin to the last visible bin and remembers the index of the largest value. It is used both for the red automatic marker and for placing a selected marker at the strongest point in the current view. The web measurement readout separately scans the full selected trace.

Why this is deliberately simpler than sub-bin interpolation

The purpose here is graphical placement on an existing plotted data point. A marker must remain attached to an actual trace bin. The measurement engine's parabolic interpolation answers a different question: the best estimate of the real peak frequency between bins.

> **Significance:** Keeping graphical peak search separate from scientific peak-frequency estimation prevents the display marker from drifting away from its actual plotted sample while still allowing a more accurate numerical measurement.

How the algorithms work together



No single algorithm provides the whole answer. The FFT creates the data, historical traces expose behavior over time, calibration makes the axes meaningful, carrier and peak algorithms locate interesting features, and measurement algorithms turn those features into values a person can interpret.

One-page summary

Algorithm	Main result	Main reason
Hann-windowed FFT	Spectrum bins	Convert IQ time samples into frequency information
Carrier detector	Occupied regions and edges	Find wide transmissions without accepting noise humps
Multi-peak detector	Distinct local peaks	Summarize several strong signals
Parabolic peak fit	Sub-bin peak frequency	Improve frequency precision without a larger FFT
Median noise floor	Typical background bin level	Resist distortion by a few strong signals
99% power bandwidth	Occupied bandwidth	Measure width from distributed signal power
Linear power sum	Total displayed power	Combine bin powers correctly
Hold and average traces	Historical extrema and mean	Reveal intermittent and long-term behavior
Calibration interpolation	dBFS-to-dBm offset	Produce honest physical power estimates
Fixed-plus-PPM correction	Corrected HackRF frequency	Compensate oscillator tuning error
UI maximum search	Strongest plotted point	Make markers fast and intuitive

Important limitations to remember

- Frequency resolution is limited by sample rate, FFT size, window, and signal-to-noise ratio.
- The measurement noise floor is the median of the whole display; it is not a locally measured noise floor beside one carrier.
- Occupied bandwidth and channel power currently include the entire displayed span, including noise and unrelated signals.
- Carrier detection requires a sustained core and rejects incomplete regions at FFT edges. Very narrow tones are therefore better represented by peak detection than carrier-region detection.
- A dBm label is only trustworthy when the live frequency and gain fall within valid calibration coverage.

Short glossary

Carrier: A continuous occupied frequency region used to carry information. **FFT:** A fast method for separating a sampled waveform into frequency components. **Bin:** One discrete frequency slot in the FFT. **Noise floor:** The background level against which signals are seen. **Hysteresis:** Using a stricter condition to start a detection and a looser condition to continue it. **MAD:** Median absolute deviation, a robust measure of variation. **dBFS:** Digital level relative to full scale. **dBm:** Power relative to one milliwatt. **PPM:** Parts per million, used for small proportional frequency errors.

End of guide. Source reviewed from the current project implementation.